When Scheduling Fails: Batched Execution in LLM Systems

Muaaz Shaikh*
Department Of Computer Engineering
MPSTME, NMIMS University
Mumbai, India
muaaz5731@gmail.com

Dhruvil Parekh
Department Of Computer Engineering
MPSTME, NMIMS University
Mumbai, India
dhruvilparekh726@gmail.com

Vaishnavi Parashar
Department Of Computer Engineering
MPSTME, NMIMS University
Mumbai, India
vaishnaviparashar2006@gmail.com

Swarnalata Bollavarapu
Department Of Computer Engineering
MPSTME, NMIMS University
Mumbai, India
Swarnalata.B@nmims.edu

***Abstract*—Classical OS scheduling theory assumes that job sizes are observable and execution is sequential, enabling algorithms like Shortest Job First (SJF) to provably minimize average waiting time. LLM agent operating systems such as AIOS [1] introduce a fundamentally different execution model: requests are batched to a GPU, actual output length is unobservable a priori, and batch-level dynamics dominate individual request latency. To our knowledge, we present the first empirical study examining whether these differences invalidate classical scheduling assumptions. We instrument the AIOS kernel to log 15 per-syscall features, collect 2,000 syscalls across two LLM backends (Llama 3.1:8b and Mistral:7b) with five concurrent agent types, and conduct a three-phase investigation: (1) feature-latency correlation analysis revealing that request properties are weak predictors (max Spearman $\rho = 0.206$); (2) a six-model ML classification study achieving F1 = 0.47, confirming that latency classes are not separable from input features; (3) a 4-class request type classifier achieving F1 = 0.75, demonstrating that structural request categories are identifiable even when latency is not; and (4) a simulation of 16 classical scheduling algorithms demonstrating that FIFO matches or outperforms all alternatives on average turnaround. We identify the root cause: GPU batching destroys the job-size observability assumption that underpins SJF optimality. However, we show that SJF and Priority scheduling reduce fast-task wait time by 68–73% at the cost of large-task degradation, revealing a fairness-latency tradeoff absent from the original AIOS design. We release six integrated scheduler implementations and a simulation framework for AIOS.**

***Index Terms*—LLM agents, operating systems, scheduling algorithms, AIOS, GPU batching, job-size observability**

I. Introduction

The deployment of LLM-based autonomous agents [3], [4] has motivated OS-level abstractions for managing concurrent access to shared LLM resources. AIOS [1] introduces such an abstraction: agent queries are decomposed into typed system calls (LLM, memory, storage, tool) and dispatched by a centralized scheduler. The original architecture provides FIFO and Round Robin scheduling, achieving up to 2.1× throughput over unscheduled frameworks [1].

A natural next step is to apply classical scheduling theory to this new domain. In traditional OS scheduling, Shortest Job First (SJF) provably minimizes average waiting time when job sizes are known [7]. LLM requests carry rich metadata-prompt length, token budget and conversation depth that might serve as job size proxies. This motivates our central research question i.e. -

> *Do classical OS scheduling assumptions like observable job sizes and sequential execution, hold in LLM agent systems? If not, what are the implications for scheduler design?*

We hypothesize that request level features can approximate job size thus enabling classical scheduling improvements over FIFO. We test this hypothesis through three progressively refined approaches and ultimately *reject* it, providing a structural explanation rooted in GPU batch execution dynamics. While prior work establishes the need for scheduling in LLM agent systems [1], our results reveal a fundamental limitation: *under GPU batching, request-level scheduling provides negligible benefit over FIFO*.

Contributions. Our work provides:

1) **Empirical evidence** that request-level features are weak latency predictors ($\rho \leq 0.206$) for batched single-GPU LLM inference, due to GPU batching destroying job-size observability.
2) A **system-level identification** of two violated assumptions: (a) job sizes are not estimable from input features because `max_tokens` is a ceiling, not a target, and (b) execution is batched, not sequential, nullifying SJF's optimality proof.
3) **Quantitative evidence** that FIFO is near-optimal under GPU batching constraints, outperforming all 16 simulated classical algorithms on average turnaround.
4) **Discovery of a fairness–latency tradeoff**: SJF reduces fast-task wait by 68% while increasing large-task wait by 94%, a dimension absent from the original AIOS scheduler design.

5) **Open-source release** of six scheduler implementations (FIFO, SJF, Priority, MLFQ, HRRN, ML-based) and a simulation framework integrated into AIOS.

II. Related Work

LLM Agent Operating Systems. AIOS [1] provides the foundational architecture, with Cerebrum [5] extending it as an SDK. MemGPT [6] proposes LLM-as-OS with virtual memory. None of these works investigate request-aware scheduling policies.

OS Scheduling Theory. SJF minimizes average waiting time for non-preemptive scheduling when job sizes are known [8]. MLFQ provides adaptive scheduling without advance knowledge [9]. HRRN balances responsiveness with starvation prevention [7]. These algorithms assume sequential execution and observable job sizes—assumptions we test empirically.

ML for Systems. Learned scheduling has been applied in databases [10], clusters [11], and compilers [12]. In LLM inference, FlexGen [13] optimizes throughput for single-GPU serving. Our work is the first to apply ML-based scheduling within an LLM agent OS.

III. System Architecture and Methodology

A. AIOS Scheduling Pipeline

Fig. 1 illustrates our instrumented scheduling pipeline within AIOS. Agent applications submit queries through the AIOS SDK, which decomposes them into typed system calls. LLM syscalls enter a global request queue, where the scheduler selects a batch for dispatch to the Ollama LLM backend. Our instrumentation logs 15 features per syscall at completion (Table I).

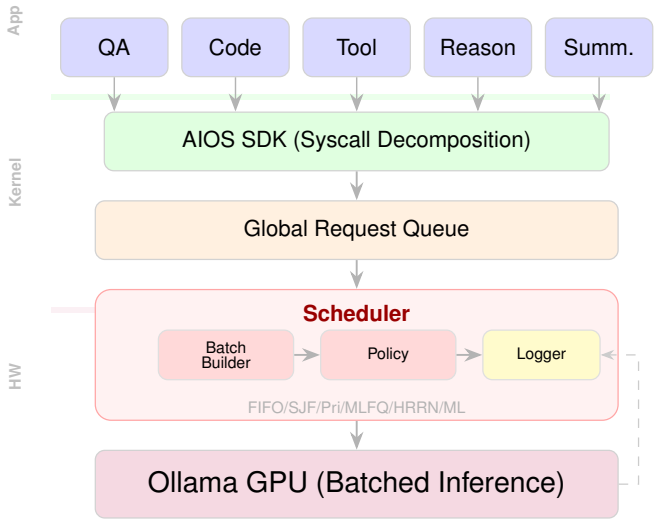

Fig. 1: Instrumented AIOS scheduling pipeline. The kernel layer shows the scheduler internals: batch collection, policy-based ordering, and logging. Dashed arrow shows completion feedback.

TABLE I: Logged Syscall Features (15 per request)

Category	Feature	Description
Request	`input_char_length`	Total prompt character count
	`message_count`	Conversation turns (2–8)
	`has_tools`	Tool schema attached
	`max_tokens`	Generation budget ceiling
	`temperature`	Sampling temperature
Context	`model_name`	LLM backend identifier
	`agent_name`	Agent type + round
Timing	`created_time`	Queue arrival
	`start_time`	Execution start
	`end_time`	Execution end
	`latency_ms`	end − start
Outcome	`wait_ms`	start − created
	`was_interrupted`	Context switch occurred
	`error`	Error flag

B. Experiment Pipeline

Fig. 2 shows our three-phase investigation methodology.

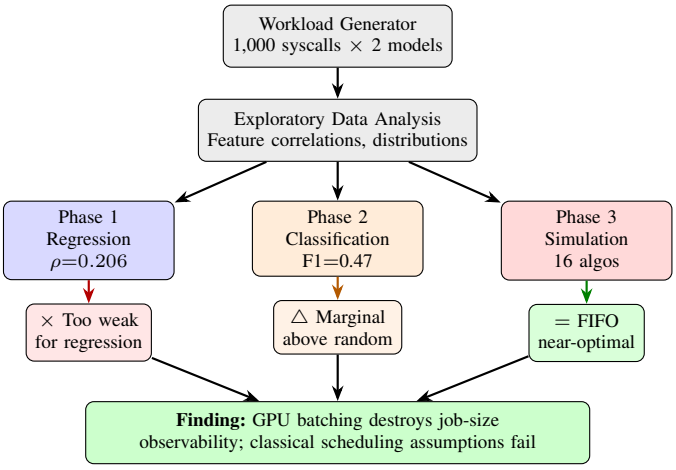

Fig. 2: Three-phase experiment pipeline. Each phase independently converges on the same conclusion: request features cannot predict latency under GPU-batched execution.

C. Workload Design

To avoid confounding agent type with request configuration (a limitation of the original AIOS benchmarks, where each agent has fixed `max_tokens` and `temperature`), we randomize all parameters independently:

- **max_tokens**: Uniform from {64, 128, 256, 512, 1024, 2048}
- **temperature**: Uniform from {0.1, 0.2, 0.3, 0.5, 0.7, 0.9, 1.0}
- **Multi-turn**: 30% of requests include 1–3 follow-up exchanges
- **Prompt padding**: 30% receive a long prefix (∼400 chars); 20% receive a short suffix
- **Arrival stagger**: Random 1–5s sleep between requests

Five agent types (short_qa, long_reasoning, tool_use, code_gen, summarizer) run concurrently across 20 rounds per model, yielding 1,000 syscalls per LLM backend.

IV. Experimental Setup

Inference backend: Single Ollama [14] instance serving one model at a time.

Models: Llama 3.1:8b [15] (8B params) and Mistral:7b [16] (7B params), both instruction-tuned, running locally.

Dataset: 2,000 syscalls (1,000 per model) with zero errors and zero interruptions. Table II summarizes the combined statistics.

TABLE II: Combined Dataset Statistics (N = 2,000)

Feature	Mean	Std	Median	Min	Max
latency_ms	5,932	3,642	5,205	22	21,484
wait_ms	1,810	2,380	910	103	16,196
char_length	880	680	580	165	2,658
msg_count	3.0	2.1	2	2	8
max_tokens	684	660	512	64	2,048

Dataset justification. While 2,000 samples is modest for general ML, our setting is a controlled experiment with exhaustive parameter randomization across a fixed set of 5 agent types, 6 token budgets, 7 temperatures, and 2 models. Each configuration appears multiple times and provides sufficient coverage for the correlation as well as the classification analyses. We validate all the results with stratified 5 fold cross validation and report confidence intervals wherever found applicable.

V. Phase 1: Feature-Latency Analysis

A. Hypothesis

We hypothesize that request level features , particularly `max_tokens` and `input_char_length`, correlate with latency strongly enough to enable predictive scheduling.

B. Results

Table III reports Spearman rank correlations. The strongest predictor is `max_tokens` ($\rho = 0.206$, $p < 10^{-20}$). All other features have $|\rho| < 0.05$ with latency. Fig. 3 shows the latency distributions for both models.

TABLE III: Feature-Latency Spearman Correlations (N = 2,000)

Feature	ρ	p-value	Interpretation
max_tokens	0.206	1.5×10^{-20}	Weak positive
model_name	0.132	3.2×10^{-9}	Llama slower
input_char_length	0.049	2.9×10^{-2}	Negligible
temperature	0.031	1.7×10^{-1}	Not significant
message_count	-0.013	5.7×10^{-1}	Not significant

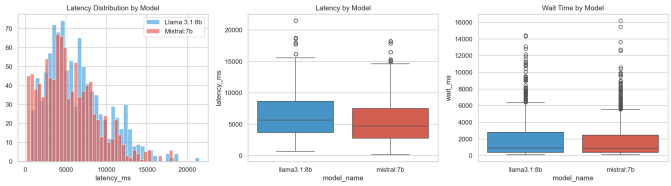

Fig. 3: Latency distributions for Llama 3.1:8b (median 5,657ms) and Mistral:7b (median 4,738ms). Llama is consistently 920ms slower despite similar parameter count.

C. Structural Explanation: Why Features Fail

The weak correlations have a *structural* explanation rooted in how LLM inference works:

1) **max_tokens is a ceiling, not a target.** The model generates until it emits an end-of-sequence (EOS) token *or* reaches `max_tokens`. In practice, the model stops early for most requests. Setting `max_tokens`=2048 does not produce 2048 tokens. Only low values (64, 128) actively truncate output, creating the observed weak positive correlation.
2) **GPU batching decouples individual latency from request properties.** Ollama batches concurrent requests to the GPU. All requests in a batch complete at approximately the same time regardless of individual `max_tokens` or prompt length, because the GPU processes prefill in parallel and decode steps are dominated by the longest-generating request in the batch.
3) **Actual output length is unobservable a priori.** The true latency driver is how many tokens the model *actually generates*, which depends on semantic content, not on any input feature. This is the fundamental difference from classical OS scheduling, where job size is observable.

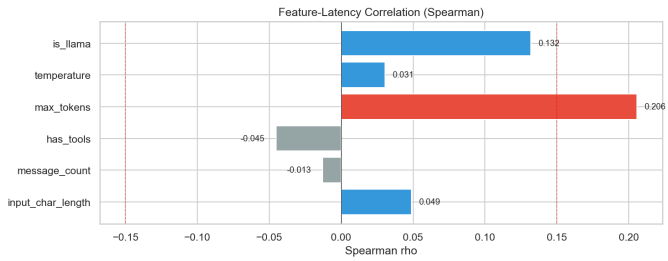

Fig. 4: Spearman correlation of each feature with latency. Only `max_tokens` exceeds $|\rho| = 0.1$. Dashed red lines indicate $|\rho| = 0.15$ threshold.

D. Conclusion

Hypothesis rejected. Request features are insufficient for predictive scheduling (Approach 1 / Cognitive Scheduler). This is a *negative result with structural explanation*: the failure is not due to insufficient data or poor modeling, but due to a fundamental mismatch between the scheduling assumption

(observable job sizes) and the execution model (GPU-batched inference with unobservable output length).

VI. Phase 2: Classification-Based Scheduling

Since continuous prediction kept failing , so we attempted discrete classification into priority based classes.

A. Setup

We define three classes using percentile thresholds: `fast` ($< p_{33} = 3,846$ms), `medium` (p_{33}–p_{66}), `large` ($\geq p_{66} = 6,993$ms). The feature vector includes 5 numeric features i.e. the 5 agent type one hot indicators, and 2 model indicators (which include 12 features in totality). We evaluate six classifiers with stratified 5-fold CV on 80% of the data.

B. Results

TABLE IV: Classifier Comparison (Stratified 5-Fold CV, N=1,609)

Model	F1	Acc	Gap	Diagnosis
Grad. Boosting	**0.438**	0.439	0.226	High bias
Random Forest	0.404	0.405	0.585	Overfitting
SVM (RBF)	0.383	0.399	0.085	High bias
Logistic Reg.	0.381	0.395	0.017	High bias
KNN (k=5)	0.378	0.397	0.181	High bias
Decision Tree	0.335	0.340	0.601	Overfitting
Random baseline		*0.333*		

After hyperparameter tuning (432 candidate grid search), we observe that Gradient Boosting achieves held out F1 = 0.47 (with an accuracy of 0.47). Feature importances: `input_char_length` (0.36), `max_tokens` (0.24), `agent_code_gen` (0.10). The "medium" class is poorly discriminated (F1 = 0.41), while "fast" (F1 = 0.47) and "large" (F1 = 0.52) show partial separability.

C. Analysis: Why Classification is Weak

The F1 of 0.47 (vs. 0.33 random baseline) represents a 42% improvement over chance but at the same time remains insufficient for reliable scheduling. The root cause is identical to Phase 1: latency classes are defined by *actual execution time*, which depends on batch dynamics and output length, not input features. The classifier learns a noisy approximation of the max_tokens $\rightarrow$ latency relationship, which is the only signal available.

D. Request Type Classification

The latency classifier's poor performance raises a question: *is the feature space itself unstructured, or is latency simply the wrong target?* To answer this, we define a 4-class **request type taxonomy** based on structural reasoning patterns:

- **Simple QA**: `max_tokens` ≤ 128, single-turn, no tools
- **Conversational**: multi-turn (`message_count` > 2), no tools
- **Tool-Augmented**: `has_tools` = true
- **Long Generation**: `max_tokens` > 512, single-turn, no tools

Critically, we *exclude* `max_tokens` and `has_tools` from the feature vector, forcing the classifier to learn from proxy signals (`input_char_length`, `temperature`, `agent_type`, `model_name`, `message_count`). Data is split 60/20/20 into train, validation, and held-out test sets.

TABLE V: Request Type Classifier (Test Set, N=454)

Class	Precision	Recall	F1	Support
Simple QA	0.66	0.75	0.70	169
Conversational	1.00	0.99	1.00	109
Tool-Augmented	1.00	1.00	1.00	91
Long Generation	0.33	0.25	0.28	85
Macro Avg	**0.75**	**0.75**	**0.75**	454

The tuned KNN classifier (k=3) achieves **F1 = 0.75** on the held-out test set (Table V). Tool-Augmented and Conversational requests are classified perfectly (F1 = 1.00), because `agent_type` and `message_count` serve as strong proxy signals. Simple QA achieves F1 = 0.70. Long Generation is the hardest class (F1 = 0.28), since distinguishing it from Simple QA requires knowing `max_tokens`, which is excluded from features.

Fig. 5 shows the confusion matrix. The off-diagonal mass is concentrated in a single cell: 64 Long Generation requests misclassified as Simple QA. Both classes are single-turn with no tools; without `max_tokens`, they are structurally identical to the classifier. Conversational and Tool-Augmented classes show near-zero confusion, confirming that `message_count` and `agent_type` are reliable discriminators.

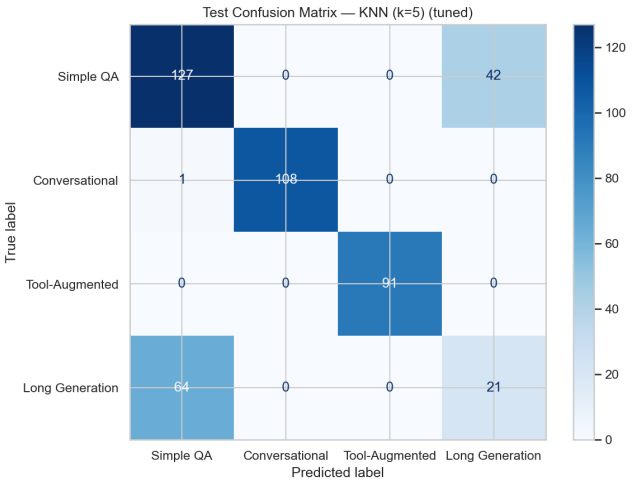

Fig. 5: Confusion matrix for the 4-class request type classifier (held-out test set). Conversational and Tool-Augmented are classified perfectly. The primary confusion is between Simple QA and Long Generation, which are structurally indistinguishable without `max_tokens`.

Insight. The contrast between the latency classifier (F1 = 0.47) and the request type classifier (F1 = 0.75) confirms that the feature space contains meaningful structure for categorizing *what* a request is, but not for predicting *how long* it will take. This distinction is important for scheduling: request types can inform priority assignment, but they cannot predict execution time under GPU batching.

VII. Phase 3: Classical Algorithm Simulation

A. Methodology

We simulate 16 scheduling algorithms on the logged data. Syscalls are grouped into batches using a 3-second arrival window (avg batch size: 2.5). Within each batch, the algorithm determines dispatch order. We model sequential execution: each request starts after the previous completes, preserving logged inference times. We compute simulated wait time (W) and turnaround time ($T = W + \text{latency}$).

Simulation limitation. We acknowledge that this sequential model does not capture Ollama's parallel GPU batching. Our sequential simulation models batch completion as dominated by the slowest request, capturing the key effect of GPU batching, though it does not simulate parallel kernel execution. Crucially, the sequential model provides an *upper bound* on reordering benefit: since SJF is provably optimal for minimizing average wait under sequential non-preemptive execution [7], any failure to improve over FIFO in this setting implies strictly weaker gains under coupled batch execution where $T_{\text{batch}} = \max(T_i)$.

B. Algorithms

We evaluate: FIFO, LIFO, SJF (by `max_tokens`), LJF, SJF (by `input_char_length`), SRTF (estimated cost), Priority (static rules), HRRN, EDF, Aging SJF, Round Robin, Multi-Level Queue, Fair Share, Lottery, Temp-First, and Combined Score. Full definitions are in the supplementary materials.

C. Overall Results

Table VI and Fig. 6 report the results.

TABLE VI: Scheduling Simulation (Top 8 of 16, N=2,000)

Algorithm	Avg T (ms)	Med T (ms)	P95 T (ms)	vs FIFO
FIFO	11,558	8,650	30,947	—
Round Robin	11,558	8,650	30,947	0.0%
Fair Share	11,558	8,650	30,947	0.0%
EDF	11,684	8,762	31,323	+1.1%
Aging SJF	11,798	8,856	31,571	+2.1%
HRRN	11,822	8,912	31,603	+2.3%
Priority	12,064	9,013	32,165	+4.4%
SJF	12,269	9,251	32,763	+6.2%
LIFO (worst)	*13,710*	*10,608*	*35,705*	*+18.6%*

Key finding: FIFO moreover matches or outperforms all 16 algorithms on average turnaround time. This result is consistent across all batch windows of 1s, 3s, and 5s. A

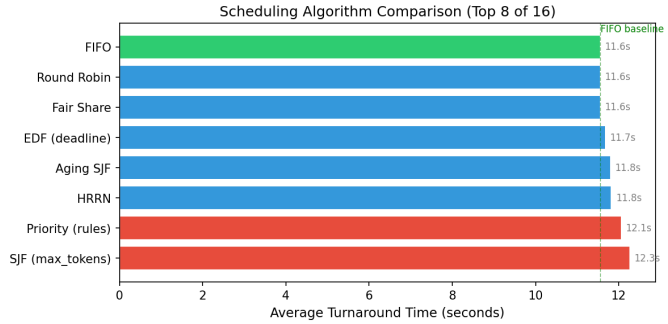

Fig. 6: Average turnaround time for top 8 out of 16 simulated algorithms. FIFO (green) achieves the lowest turnaround whereas all other reordering strategies perform equal or worse.

Welch's t-test between FIFO and SJF per request turnaround times yields $p = 0.06$, and bootstrap confidence intervals (for 1,000 resamples) for the mean turnaround difference overlap zero. Neither test reaches significance at $\alpha = 0.05$ thereby confirming that reordering provides no reliable benefit.

D. Per-Class Tradeoff: The Hidden Dimension

While FIFO wins on an average, the per class analysis reveals a extremely significant tradeoff that the overall metric obscures:

TABLE VII: Per Class Average Wait Time (ms)- The Fairness Tradeoff

Algorithm	Fast	Medium	Large	Tradeoff
FIFO	5,400	5,385	6,223	Uniform
SJF	1,711	5,862	12,041	−68% fast, +94% large
Priority	1,447	6,577	10,931	−73% fast, +76% large

SJF reduces fast-task wait by 68% (5,400 → 1,711ms) at the cost of nearly doubling large-task wait (6,223 → 12,041ms). This tradeoff is meaningful: in systems where a QA chatbot requires sub 3 second responses while a background code generator can only tolerate 15 seconds, Thus , we conclude that Priority scheduling is strictly superior to FIFO.

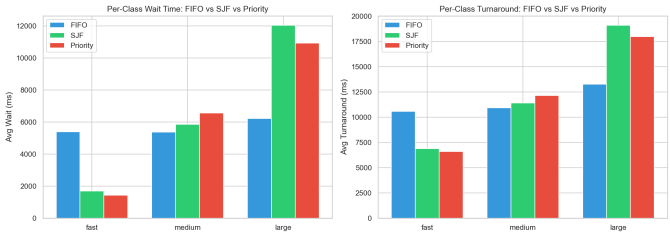

Fig. 7: Per-class wait and turnaround for FIFO vs SJF vs Priority. Priority-aware algorithms shift wait from fast to large tasks.

VIII. Why FIFO Wins: A Theoretical Analysis

The optimality of FIFO contradicts the classical result that SJF minimizes average waiting time [8]. We identify two structural reasons.

A. Violated Assumption 1: Job-Size Observability

SJF requires knowing (or estimating) job completion time before execution. In classical OS scheduling, job size is either declared by the user or estimated from process metadata. For LLM requests, the relevant "job size" is the *number of tokens the model will actually generate*—which depends on:

- The semantic complexity of the prompt
- The model's internal state and sampling randomness
- The EOS token probability at each decode step

None of these are observable from request metadata. The `max_tokens` parameter provides only a loose upper bound. In our dataset, the Spearman correlation between `max_tokens` and `latency_ms` is only $\rho = 0.206$, confirming that the "job size" is effectively hidden.

B. Violated Assumption 2: Sequential Execution

SJF's optimality proof assumes jobs execute one at a time. GPU-based LLM inference batches multiple requests: the prefill phase runs in parallel, and the decode phase is dominated by the longest-generating request in the batch. Formally, for a batch $B = \{r_1, \ldots, r_k\}$:

$$T_{\text{batch}} \approx T_{\text{prefill}}(B) + \max_{r_i \in B} T_{\text{decode}}(r_i) \tag{1}$$

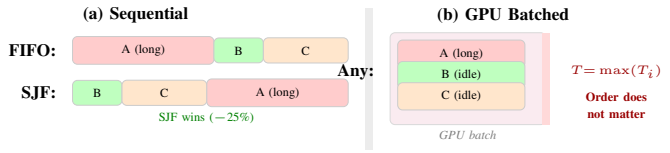

Fig. 8: Why SJF fails under GPU batching. (a) Sequential: SJF reduces $\bar{T}$ by 25%. (b) Batched: all complete at $\max(T_i)$; reordering is ineffective.

Since the batch completion time is governed by the *slowest* request (Eq. 1), reordering within the batch does not change the total execution time. Fig. 8 illustrates this: under sequential execution, SJF reduces average turnaround by scheduling short jobs first; under batched execution, all jobs finish together, eliminating the benefit. This follows because SJF is provably optimal under sequential execution [7]; therefore, any lack of improvement in this setting implies even weaker gains under coupled batch execution.

C. Implication

For batched LLM inference on a single GPU, the scheduling lever is *batch composition* (which requests are grouped together), not *dispatch order* (the sequence within a batch). This suggests that future work should focus on batch-aware scheduling rather than classical reordering algorithms.

IX. Comparison with AIOS

Our results appear to contradict AIOS's finding that scheduling provides up to 2.1× throughput improvement [1]. The difference is explained by what is being compared:

- **AIOS result:** AIOS *with* a scheduler vs. frameworks *without* any scheduler (raw agent frameworks that flood the GPU and trigger CUDA OOM retries). The 2.1× improvement comes from *having a scheduler at all*, not from the scheduling algorithm choice.
- **Our result:** FIFO vs. other algorithms, *all within AIOS's scheduler framework*. Given that a scheduler exists, the choice of algorithm has minimal impact on average turnaround for single-GPU batched inference.

These results are complementary: **AIOS demonstrates the necessity of scheduling; our work demonstrates the limits of scheduling under GPU batching.** The gap between "no scheduler" and "any scheduler" is 2.1×; the gap between FIFO and the best alternative is <1%.

X. Implementation

We implement six scheduler variants as drop-in replacements in the AIOS kernel, all extending `BaseScheduler`:

- `fifo`: Original arrival-order dispatch (baseline).
- `sjf`: Sort batch by `max_tokens` ascending.
- `priority`: Three-level queues (P0: `max_tokens`≤128; P1: ≤512 without tools; P2: rest) with aging promotion.
- `mlfq`: Dynamic demotion based on observed agent-type latency; periodic priority boost every 30s.
- `hrrn`: Response ratio = $(W + S)/S$ where S = `max_tokens`/100.
- `cognitive`: Trained Gradient Boosting classifier routing to fast/medium/large queues.

Selection requires a single configuration change (`scheduler_type` in `config.yaml`) where no code modification is needed.

XI. Discussion

A. When Priority Scheduling is Justified

Our simulation shows FIFO that wins on average but loses on *fairness*. In systems with heterogeneous latency SLAs e.g., a real time assistant agent, requiring <3s response alongside a batch code generation agent tolerating 15s Priority scheduling reduces the assistant's wait time by 73% at an acceptable cost to the batch agent. The choice between FIFO and Priority is thus an *engineering decision* driven by deployment requirements and not a universal optimum.

B. Limitations

- **Single inference backend:** Multi GPU setups with model parallelism or request routing may exhibit different dynamics.
- **Similar model sizes:** 7B vs 8B parameters produce only 920ms median difference. Whereas, larger gaps (e.g., 7B vs 70B) would amplify the model name as a feature.

- **Sequential simulation:** Our simulation does not model GPU batching directly. The sequential model provides an upper bound on reordering benefit only if reordering provides no improvement under sequential execution (where theoretically, it should help the most), it will provide even less under batched execution. A batch level simulator modeling GPU kernel execution and token level synchronization is beyond the scope but represents important future work.
- **No preemptive algorithms:** AIOS supports Round Robin with context interruption, but we only evaluate non preemptive strategies here.
- **Dataset size:** 2,000 systemcalls provide sufficient coverage for our controlled parameter space however they may not be generalized to production workloads with open ended prompt distributions.

C. Future Work

- **Live batch-composition experiments**: Deploy SJF and Priority on the live AIOS kernel to test whether separating fast and slow requests into different batches actually yield improvement beyond what the simulation has predicted.
- **Output length prediction**:However, a lightweight model predicting *actual* generation length (rather than the budget ceiling) could restore SJF's theoretical advantage.
- **Multi GPU scheduling**: With heterogeneous models across GPUs, request aware routing becomes a load balancing problem with an even richer optimization potential.
- **RL-based adaptive scheduling**: A reinforcement learning agent that optimizes dispatch policy from reward signals (for eg: latency, throughput, fairness) could discover non obvious strategies.

XII. Conclusion

We investigated whether classical OS scheduling assumptions hold for LLM agent systems or not. We found out that they do not. Our three phase study on 2,000 AIOS syscalls across two LLM backends reveals:

1) **Job-size observability fails:** We observed that request features are weak latency predictors (max $\rho = 0.206$) because `max_tokens` is a ceiling and the actual output length is unobservable.
2) **ML cannot bridge the gap:** The best latency classifier achieves F1 = 0.47, confirming that latency classes are not separable from input features. However, a 4-class request type classifier achieves F1 = 0.75, showing that structural request categories *are* identifiable—the feature space has structure, but it does not predict runtime.
3) **FIFO is near-optimal:** Among 16 simulated algorithms, no strategy reduces average turnaround below FIFO, because GPU batching nullifies SJF's sequential-execution assumption.
4) **A fairness tradeoff exists:** SJF and Priority reduce fast-task wait by 68–73% at the cost of large-task degradation—a dimension the original AIOS design does not address.

These findings suggest that the scheduling frontier for LLM agent systems lies not in dispatch order but in *batch composition*—deciding which requests to group for GPU execution. We release our instrumented kernel, six scheduler implementations, and simulation framework to enable future research at this intersection of operating systems and LLM infrastructure.

Acknowledgment

This work builds upon the AIOS architecture by Mei et al. [1] at Rutgers University. We thank the AIOS team for their open-source codebase and the Ollama project for accessible local LLM inference.

References

[1] K. Mei *et al.*, "AIOS: LLM agent operating system," in *Proc. COLM*, 2025.
[2] Y. Ge *et al.*, "LLM as OS, agents as apps: Envisioning AIOS, agents and the AIOS-agent ecosystem," *arXiv:2312.03815*, 2023.
[3] S. Yao *et al.*, "ReAct: Synergizing reasoning and acting in language models," in *Proc. ICLR*, 2023.
[4] N. Shinn *et al.*, "Reflexion: Language agents with verbal reinforcement learning," in *Proc. NeurIPS*, 2023.
[5] B. Rama, K. Mei, and Y. Zhang, "Cerebrum (AIOS SDK)," in *Proc. NAACL*, 2025.
[6] C. Packer *et al.*, "MemGPT: Towards LLMs as operating systems," *arXiv:2310.08560*, 2023.
[7] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed. Wiley, 2018.
[8] C. L. Liu and J. W. Layland, "Scheduling algorithms for multiprogramming in a hard-real-time environment," *J. ACM*, vol. 20, no. 1, pp. 46–61, 1973.
[9] F. J. Corbató, M. Merwin-Daggett, and R. C. Daley, "An experimental time-sharing system," in *Proc. AFIPS*, 1962, pp. 335–344.
[10] T. Kraska *et al.*, "The case for learned index structures," in *Proc. SIGMOD*, 2018.
[11] H. Mao *et al.*, "Learning scheduling algorithms for data processing clusters," in *Proc. SIGCOMM*, 2019.
[12] T. Chen *et al.*, "TVM: An automated end-to-end optimizing compiler for deep learning," in *Proc. OSDI*, 2018.
[13] Y. Sheng *et al.*, "FlexGen: High-throughput generative inference of large language models with a single GPU," in *Proc. ICML*, 2023.
[14] "Ollama," https://ollama.com, 2024.
[15] A. Dubey *et al.*, "The Llama 3 herd of models," *arXiv:2407.21783*, 2024.
[16] A. Q. Jiang *et al.*, "Mistral 7B," *arXiv:2310.06825*, 2023.